

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

Equipe 404 Not Found

Relatório da Sprint 1

Todos os itens da rubrica foram avaliados; este documento lista somente itens em que foram encontradas falhas ou inconformidades.

Quando um item não aparece a seguir, não foi registrada falha objetiva para ele na inspeção realizada.

Engenharia de Software II

Item avaliado	Falha encontrada
1.1 README principal	Não há instruções de execução suficientes para reproduzir a aplicação. O README também contém links defasados para 2dsm ABP/app - aluno, pasta que não existe na estrutura atual, e a rota Swagger documentada parece incorreta.
1.2 README das pastas principais	Não foram encontrados README.md nas pastas principais 2dsm_ABP/frontend, 2dsm_ABP/backend ou 2dsm_ABP/init/database. Assim, as responsabilidades dessas pastas não ficam explicadas dentro dos próprios módulos.
1.3 Estrutura do projeto	A estrutura real usa 2dsm_ABP e init, enquanto o README descreve 2dsm ABP, app - aluno e database. Essa divergência prejudica a coerência entre organização real e documentação.
2.3 Definition of Done (DoD)	O DoD documentado exige, por exemplo, carregar menus a partir do banco e executar frontend/backend/banco via Docker Compose, mas a Sprint 1 declara e implementa fluxo com dados mockados, sem integração real com banco/API. Portanto, os itens entregues não cumprem integralmente o DoD definido.
3.1 Diagrama de Casos de Uso	O diagrama possui relacionamentos include/extend excessivos e cruzados, associações longas e difíceis de interpretar e limite do sistema pouco claro. Isso gera inconsistências conceituais e baixa legibilidade.
3.2 Coerência entre documentação e implementação	A documentação geral menciona evidências documentais, persistência, logs, JWT e rotas administrativas, mas a Sprint 1 implementa principalmente um chatbot frontend mockado. A seção de evidências da Sprint 1 ainda contém placeholders sem links de demo ou imagens.

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

4.1 Estratégia de branches	Foram encontradas main, dev e docs, mas não foram identificadas branches de feature/fix ativas ou remotas na inspeção local.
4.2 Distribuição temporal dos commits	Há concentração relevante de commits nos dias 04/05/2026 e 05/05/2026, véspera/data da entrega, incluindo frontend, diagrama, documentação e merge. Isso indica acumulação no fim da sprint.
4.3 Participação dos membros	A distribuição de commits é desequilibrada: Luiza-Manchini aparece com 27 commits, Felipe/Felipe1781/felipefmac com 13 somados, conta institucional com 6, Ariana/arianaferresan com 3, William com 2 e Lucas com 1. Pedro e Eloah constam como responsáveis nas tarefas, mas não aparecem no shortlog local consultado.

Desenvolvimento Web II

Item avaliado	Falha encontrada
2.1 Estrutura em camadas	O backend separa controllers, routes e config, mas não possui services nem repositories. Consultas SQL e regras ficam diretamente nos controllers, reduzindo a separação de responsabilidades.
2.2 Organização das rotas	A rota publica definida para opções exige /api/options/:id, enquanto o controller possui lógica para raiz sem id. Além disso, existe rota pública destrutiva DELETE /api/deve/deletall sem proteção, o que quebra o padrão esperado de rotas públicas.
3.1 Uso de variáveis de ambiente	Não há .env.example no repositório para documentar as variáveis necessárias. docker compose config falhou porque PORT, DB_NAME, DB_USER e DB_PASSWORD não estavam definidos, gerando porta inválida.
4.2 Organização lógica do frontend	Não há camada de services para acesso a API nem contexts. O hook useChatFlow concentra fluxo, dados, textos extensos, regras de navegação, formulário simulado e avaliação, dificultando manutenção e futura integração com backend.

Banco de Dados Relacional

Item avaliado	Falha encontrada
---------------	------------------

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

1.1 Conexão PostgreSQL	Não foi possível confirmar a conexão da aplicação ao PostgreSQL. O docker compose config falhou por ausência de variáveis de ambiente, e o fluxo principal do frontend não consome o backend/banco.
1.2 Persistência correta	Dados relevantes do chatbot estão hardcoded no frontend, especialmente em useChatFlow.ts. O envio de dúvida e a avaliação de satisfação apenas simulam sucesso e não persistem no banco.
2.1 Qualidade dos comandos SQL	O schema cobre entidades básicas, mas não modela documentos, chunks e metadados como tabelas próprias, apesar de esses conceitos existirem na documentação do produto. O seed SQL também é mínimo e não sustenta os fluxos completos demonstrados no frontend.
2.2 Otimização inicial	Não foram encontrados índices explícitos para consultas prováveis, como navigation_nodes.parent_id, navigation_nodes.is_active ou filtros de contatos por status/data. Há redundância entre dados no banco seed e conteúdo hardcoded no frontend.

Técnicas de Programação I

Item avaliado	Falha encontrada
1.1 Aplicação funcional	Não foi possível comprovar execução funcional. npm run build em frontend e backend falhou neste ambiente por erro local do Node/WSL 1, e Docker Compose não validou por falta do .env.
1.2 Execução containerizada	O docker-compose.yml define banco e backend, mas o serviço frontend está comentado. Assim, a aplicação completa não sobe com comando único pelo Compose atual.
2.1 Tipagem TypeScript	Apesar de haver interfaces e strict nos tsconfigs, o frontend usa strings soltas para tipos de ChatItem e acessos com non-null assertion em vários pontos do renderItem. Isso reduz a robustez da tipagem.
2.2 Uso de conceitos de POO	Não há uso relevante de classes, encapsulamento ou abstrações orientadas a objeto. A maior parte da lógica está em funções, hooks e controllers procedurais; isso atende parcialmente ao estilo React/Express, mas é fraco para o critério específico de POO.

